Adaptive Runtime Federation in Discovery-based Querying

Anonymous Author(s)

Abstract

Your abstract text goes here. Keep it concise, focused on the core database or data management challenge, your approach, and key experimental findings.

CCS Concepts

• **Information systems** → **Data management systems**.

Keywords

databases, query processing, distributed systems

1 Introduction

2 Related Work

3 Methodology

3.1 Preliminaries

Let $\mathcal{U}$, $\mathcal{B}$, $\mathcal{L}$ and $\mathcal{V}$ be pairwise disjoint sets of IRIs, blank nodes, literals and variables. An *RDF triple* is an element of $\mathcal{T} = (\mathcal{U} \cup \mathcal{B}) \times \mathcal{U} \times (\mathcal{U} \cup \mathcal{B} \cup \mathcal{L})$, and an *RDF graph* a finite subset of $\mathcal{T}$. A *triple pattern* is an element of $\mathcal{P} = (\mathcal{U} \cup \mathcal{V}) \times (\mathcal{U} \cup \mathcal{V}) \times (\mathcal{U} \cup \mathcal{L} \cup \mathcal{V})$ and a *basic graph pattern* (BGP) a finite set of triple patterns.

A *solution mapping* is a partial function $\mu : \mathcal{V} \to \mathcal{U} \cup \mathcal{B} \cup \mathcal{L}$; we write $\mu_\emptyset$ for the one with empty domain, Ω for a set of solution mappings, and $\bowtie$ for their join. Following Pérez et al. [9], for a triple pattern t we write $\mu(t)$ for the triple obtained by replacing the variables in t according to μ, and $\mathrm{var}(t)$ for the variables occurring in t. A triple pattern evaluates over a graph G to

$$[\![t]\!]_G \;=\; \{\, \mu \mid \mathrm{dom}(\mu) = \mathrm{var}(t) \text{ and } \mu(t) \in G \,\},$$

and evaluating a BGP B distributes over its patterns,

$$[\![B]\!]_G \;=\; \bowtie_{t \in B} [\![t]\!]_G, \qquad [\![\emptyset]\!]_G = \{\mu_\emptyset\},$$

so the empty BGP evaluates to the identity for $\bowtie$. For a BGP B we write

$$\mu(B) \;=\; \{\, \mu(t) \mid t \in B \,\}$$

for the set of triples obtained by applying μ to every pattern of B; patterns that μ maps to the same triple contribute it once.

As triple patterns contain no blank nodes, this evaluation coincides with basic graph pattern matching in SPARQL [6]. Writing $\mathrm{var}(B)$ for the variables of B, a mapping is a solution of B exactly when it binds all variables of B and every triple it produces from B occurs in G,

$$\mu \in [\![B]\!]_G \iff \mathrm{dom}(\mu) = \mathrm{var}(B) \text{ and } \mu(B) \subseteq G. \tag{1}$$

In other words, $\mu(B)$ is the set of triples of G from which the solution μ is obtained.

Blank nodes are existential variables scoped to the document that serialises them [3]: no two sources share one, and none can be referenced in a later request [10]. As a result, a query decomposed over several sources can lose answers that join on a blank node within one source [14]. We exclude them.

Assumption 1 (Skolemised sources)**.** *Every source publishes a graph free of blank nodes,* $\mathrm{data}(d) \subseteq \mathcal{U} \times \mathcal{U} \times (\mathcal{U} \cup \mathcal{L})$*. Blank nodes are replaced by Skolem IRIs [3], and a source republishing data preserves those minted by its origin.*

FedQPL assumes only that members use pairwise disjoint blank nodes [2]. This is exactly what breaks aggregation: a composite republishing a document must rename its blank nodes apart, so they no longer denote the nodes its origin published. Skolem IRIs survive republication.

3.2 Query Delegation in Discovery Settings

Dynamically integrating conjunctive sub-queries during federated query processing in discovery-based querying at runtime is difficult due to the unbounded nature of the query execution. To show this, we first introduce a sufficient condition for correctly delegating conjunctive queries in the traditional federated case. Then we will show how deciding whether we can correctly delegate these queries in the discovery-based querying case is impossible when not all sources have been discovered.

First we define our discovery-based federated query setting in Definition 1 and 2.

Definition 1 (Discovery setting)**.** A *discovery setting* is a tuple $W = \langle \mathbb{D}, \mathrm{data}, \mathcal{D}_0, \mathrm{disc} \rangle$: a universe of sources $\mathbb{D}$, a map $\mathrm{data} : \mathbb{D} \to 2^{\mathcal{T}}$ giving the graph each one holds, a set of seeds $\mathcal{D}_0 \subseteq \mathbb{D}$, and a *discovery function* disc assigning to every BGP B a map $\mathrm{disc}_B : \mathbb{D} \times 2^{\mathcal{T}} \to 2^{\mathbb{D}}$, where $\mathrm{disc}_B(d, \mathrm{data}(d))$ are the sources that a retrieved d exposes while B is evaluated.

Definition 2 (Reachable set, reference answer)**.** Iterating discovery from the seeds, let $\mathcal{D}_{i+1} := \mathcal{D}_i \cup \bigcup_{d \in \mathcal{D}_i} \mathrm{disc}_B\big(d, \mathrm{data}(d)\big)$. The *reachable set* is $\mathrm{reach}_B(W) := \bigcup_{i \geq 0} \mathcal{D}_i$, and the *reference answer* to B is $[\![B]\!]_{G_{\mathrm{reach}_B(W)}}$. Correctness of a delegation is always measured against this reference answer, never against the universe $\mathbb{D}$: sources that are never reached cannot contribute to it.

Given this definition, we recover the traditional federated query processing setting, in which all sources are known beforehand, by taking $\mathcal{D}_0 = \mathbb{D}$ and $\mathrm{disc}_B \equiv \emptyset$.

Next we define an operator to determine the *relevance* of a source for a query execution setting. Relevance records, for each triple pattern and each source, whether that source produces matches. An example of such an operator is the usage of ASK queries to determine exclusive groups in FedX [12].

Definition 3 (Source relevance)**.** Fix a setting W and a BGP B, and write $\mathcal{D} := \mathrm{reach}_B(W)$ for its reachable set; all that follows is relative to $\mathcal{D}$. We write $[\![t]\!]_d$ for the evaluation of a triple pattern t over the data of source d. The *relevance indicator* $\Delta : \mathcal{P} \times \mathcal{D} \to \{0, 1\}$ is

$$\Delta(t, d) \;=\; \begin{cases} 1, & \text{if } [\![t]\!]_d \neq \emptyset, \\ 0, & \text{otherwise,} \end{cases}$$

and $\mathcal{D}_t := \{\, d \in \mathcal{D} \mid \Delta(t, d) = 1 \,\}$ denotes the *relevant sources* of t.

Thus, relevance means that a data source returns answers for a given triple pattern. Next, we introduce a definition of *correct* delegation of conjunctive queries to a set of sources.

Definition 4 (Delegation)**.** Let B be a BGP, $B' \subseteq B$ the delegated fragment, and $S \subseteq \mathcal{D}$ the (only) sources to which B' is sent as a single conjunctive request, where as in Definition 3, $\mathcal{D} = \mathrm{reach}_B(W)$. Write $G_A := \bigcup_{d \in A} \mathrm{data}(d)$ for $A \subseteq \mathcal{D}$. The delegated plan obtains for B'

$$R \;:=\; \Big(\bigcup_{d \in S} [\![B']\!]_{\mathrm{data}(d)} \Big) \;\cup\; [\![B']\!]_{G_{\mathcal{D}\setminus S}},$$

and the delegation is *correct* if

$$R \;\bowtie\; [\![B \setminus B']\!]_{G_{\mathcal{D}}} \;=\; [\![B]\!]_{G_{\mathcal{D}}}.$$

This definition follows the federation model of FedQPL [2]. The following sufficient condition is the one behind the exclusive groups of FedX [12]. A sub-query may be sent as a whole to a source if no other source is relevant to any of its patterns. Previous work [13] proves this sound and complete when relevance is decided by the predicates a source contains. Like the ASK queries of FedX, we instead decide relevance by the data a source holds (Definition 3). Theorem 1 below shows that even this condition cannot be checked while discovery is incomplete.

Proposition 1 (Correct delegation)**.** *Let W be a* federated *setting, that is* $\mathrm{disc}_B \equiv \emptyset$*, so that $\mathcal{D} = \mathrm{reach}_B(W) = \mathcal{D}_0$ is known in advance. Let $t_1, \ldots, t_n$ be a set of triple patterns forming a BGP B, $B' \subseteq B$ a sub-BGP, and let $d \in \mathcal{D}$. For d we define $\mathcal{D}' = \mathcal{D} \setminus \{d\}$ the set containing all other sources. Then we can correctly delegate B' to d if* $\bigcup_{t_i \in B'} \{\, d' \in \mathcal{D}' \mid \Delta(t_i, d') = 1 \,\} = \emptyset$

The proof is in Appendix A. It rests on the observation that a single triple pattern distributes over a union of sources, as matching one pattern involves no join, whereas a BGP does not.

Proposition 1 gives us a sufficient condition for delegating conjuctive sub-queries in the case where the reachable set $\mathcal{D}$ is known in advance. However, in discovery-based querying the sources are discovered at runtime, goverened by some discovery process which returns during the query execution. As such, determining if we can correctly delegate depends on the *state* of the discovery process, which is defined in Definition 5.

Definition 5 (Discovery state)**.** The state of a discovery setting $W = \langle \mathbb{D}, \mathrm{data}, \mathcal{D}_0, \mathrm{disc} \rangle$ at time T is the tuple

$$W(T) \;=\; \langle \mathbb{D}, \mathrm{data}, \mathcal{D}_0, \mathrm{disc}, \mathcal{D}(T) \rangle,$$

the setting W augmented with $\mathcal{D}(T)$, the set of sources that data has been called on by time T, subject to

$$\mathcal{D}_0 \;\subseteq\; \mathcal{D}(T) \;\subseteq\; \mathcal{D}(T') \;\subseteq\; \mathrm{reach}_B(W) \qquad \text{for all } T \le T',$$

i.e. the seeds are discovered from the start, discovery only ever adds sources, and it never adds one outside what disc can reach. For any given $W(T)$ the discovery *frontier* is

$$F(T) \;=\; \Big(\bigcup_{d_f \in \mathcal{D}(T)} \mathrm{disc}\big(d_f, \mathrm{data}(d_f)\big) \Big) \setminus \mathcal{D}(T),$$

the sources that are discovered but do not yet have data called on them. Discovery is complete when $F(T) = \emptyset$.

Next, we introduce the concept of an observation of an engine of the discovery state in Definition 6, which denotes what the engine actually can *know* about its discovery setting at given time T.

Definition 6 (Observationally equivalent states)**.** A discovery state $W(T)$ records the whole world, but a query engine at time T has access only to the *observation*

$$O(T) \;=\; \big\langle\, \mathcal{D}(T),\; \mathrm{data}|_{\mathcal{D}(T)} \,\big\rangle,$$

the sources it has retrieved together with their contents. Two states $W_1(T)$ and $W_2(T)$ are *observationally equivalent*, written $W_1(T) \sim W_2(T)$, when $O_1(T) = O_2(T)$. Note that $F(T)$ is computable from $O(T)$ and disc: the extent of the frontier is observable, its contents are not. A property of states is *determined at T* when all states observationally equivalent to $W(T)$ agree on it; an engine at time T can establish a property only if it is determined at T.

Theorem 1 (Correct delegation is undetermined under incomplete discovery)**.** *Let $W(T)$ be a discovery state for which $F(T) \neq \emptyset$. Then there exist a BGP B, a sub-BGP $B' \subseteq B$, a set of sources $S \subseteq \mathcal{D}(T)$, and two states $W_1(T)$ and $W_2(T)$, both observationally equivalent to $W(T)$, such that B' is correctly delegated to S in $W_1(T)$ but not in $W_2(T)$. Since the engine's input is $O(T)$ and all three states share it, no procedure can decide whether the delegation is correct.*

Proof. We construct two states $W_1(T)$ and $W_2(T)$, both observationally equivalent to $W(T)$, in which B' is delegated to S correctly and incorrectly respectively. The two agree on $\mathcal{D}_0$, disc and $\mathcal{D}(T)$, and on data over $\mathcal{D}(T)$; they differ only beyond the frontier, which is non-empty by hypothesis.

Take $\mathcal{D}(T) = S = \{d_1\}$ and

$$B' \;=\; B \;=\; \big\{\, (?s, p_1, ?o_1),\; (?s, p_2, ?o_2) \,\big\},$$

$$\mathrm{data}(d_1) \;=\; \big\{\, (s_1, p_1, o_1),\; (s_1, p_2, o_2),\; (s_2, p_1, o_3) \,\big\}.$$

Fix some $d^* \in F(T)$, and let every reachable source other than d_1 and d^* hold no data. The two states are then obtained by setting

$$\mathrm{data}(d^*) \;=\; \begin{cases} \big\{\, (s_3, p_2, o_4) \,\big\} & \text{in } W_1(T), \\ \big\{\, (s_2, p_2, o_4) \,\big\} & \text{in } W_2(T). \end{cases}$$

Since $d^* \notin \mathcal{D}(T)$, both states induce the same observation, and so $W_1(T) \sim W_2(T) \sim W(T)$ holds. The reachable set differs between the two, so we write G_i for $G_{\mathrm{reach}_B(W_i)}$, the union of all data reachable in $W_i(T)$. As only d_1 and d^* hold data,

$$G_1 = \mathrm{data}(d_1) \cup \big\{\, (s_3, p_2, o_4) \,\big\}, \qquad G_2 = \mathrm{data}(d_1) \cup \big\{\, (s_2, p_2, o_4) \,\big\}.$$

Write

$$\mu_1 = \{?s \mapsto s_1,\; ?o_1 \mapsto o_1,\; ?o_2 \mapsto o_2\},$$

$$\mu_2 = \{?s \mapsto s_2,\; ?o_1 \mapsto o_3,\; ?o_2 \mapsto o_4\}.$$

Neither added triple matches $(?s, p_1, ?o_1)$, so $[\![B']\!]_{\mathrm{data}(d^*)} = \emptyset$ in both states. By Definition 4 we therefore have

$$R_1 \;=\; R_2 \;=\; [\![B']\!]_{\mathrm{data}(d_1)} \;=\; \{\mu_1\}.$$

Since $B \setminus B' = \emptyset$ we have $[\![B \setminus B']\!]_{G_i} = \{\mu_\emptyset\}$, the join identity, so the correctness condition reduces to $R_i = [\![B]\!]_{G_i}$.

In $W_1(T)$ the added triple has subject s_3, which no p_1-triple mentions, so it contributes no solution and

$$[\![B]\!]_{G_1} \;=\; \{\mu_1\} \;=\; R_1 :$$

the delegation is correct. In $W_2(T)$ the added triple joins with $(s_2, p_1, o_3) \in \text{data}(d_1)$ on $?s$, so

$$[\![B]\!]_{G_2} = \{\mu_1, \mu_2\} \neq R_2 :$$

the delegation is not correct.

The triple at d^* yields no solution on its own, and (s_2, p_1, o_3) yields none within d_1; only their join does. A single conjunctive request to d_1 cannot produce it, and neither can one to d^*, so the solution is lost precisely because the join crosses the frontier. As $W_1(T)$ and $W_2(T)$ present the engine with the same observation $O(T)$ while disagreeing on correctness, no procedure reading only $O(T)$ can decide it. □

Lemma 1 (An empty frontier certifies complete discovery)**.** *$F(T) = \emptyset$ if and only if $\mathcal{D}(T) = \text{reach}_B(W)$.*

Proof. ($\Rightarrow$) The inclusion $\mathcal{D}(T) \subseteq \text{reach}_B(W)$ holds by Definition 5. For the converse, $F(T) = \emptyset$ says exactly that

$$\text{disc}\big(d_f, \text{data}(d_f)\big) \subseteq \mathcal{D}(T) \qquad \text{for every } d_f \in \mathcal{D}(T),$$

i.e. that $\mathcal{D}(T)$ is closed under disc; and $\mathcal{D}_0 \subseteq \mathcal{D}(T)$, again by Definition 5. By induction, if $\mathcal{D}_i \subseteq \mathcal{D}(T)$ then

$$\mathcal{D}_{i+1} = \mathcal{D}_i \cup \bigcup_{d \in \mathcal{D}_i} \text{disc}\big(d, \text{data}(d)\big) \subseteq \mathcal{D}(T)$$

by closure, so $\text{reach}_B(W) = \bigcup_{i \geq 0} \mathcal{D}_i \subseteq \mathcal{D}(T)$.

($\Leftarrow$) If $d \in \text{reach}_B(W)$ then $d \in \mathcal{D}_i$ for some i, hence

$$\text{disc}\big(d, \text{data}(d)\big) \subseteq \mathcal{D}_{i+1} \subseteq \text{reach}_B(W).$$

Thus $\text{reach}_B(W)$ is closed under disc, and $F(T) = \emptyset$ follows. □

{TODO: Don't like the text here} is what makes the frontier useful: $F(T) = \emptyset$ is a condition the engine can test, whereas $\mathcal{D}(T) = \text{reach}_B(W)$ is a statement about a set it cannot inspect. Together with Theorem 1 it fixes exactly when delegation can be established at all.

Corollary 1 (Delegation requires complete discovery)**.** The correctness of every delegation of a sub-BGP $B' \subseteq B$ to a set $S \subseteq \mathcal{D}(T)$ is determined at T if and only if $F(T) = \emptyset$.

Proof. ($\Leftarrow$) Let $F(T) = \emptyset$ and let $W_1(T), W_2(T)$ be observationally equivalent to $W(T)$. As $F(T)$ is computable from $O(T)$, both have an empty frontier, so $\mathcal{D} = \text{reach}_B(W) = \mathcal{D}(T)$ in both by Lemma 1. Then $G_{\mathcal{D}} = \bigcup_{e \in \mathcal{D}(T)} \text{data}(e)$ agrees in both, and since $S \subseteq \mathcal{D}(T) = \mathcal{D}$, so do $G_{\mathcal{D} \setminus S}$ and $\text{data}(d)$ at each $d \in S$. Every term of the condition of Definition 4 therefore agrees, and with it the verdict.

($\Rightarrow$) By contraposition. If $F(T) \neq \emptyset$, then Theorem 1 supplies a B', an S, and two states observationally equivalent to $W(T)$ that disagree on the correctness of delegating B' to S. That delegation is not determined at T. □

3.3 Authority Claims

From Corollary 1, runtime federation in discovery-based querying is impossible until every source is discovered. With source counts in the thousands {TODO: cite here}, this is infeasible.

However, while classical query semantics require all data to be considered in query execution, this is not always an intuitive requirement. Consider a federation of product catalogues, in which a manufacturer publishes the specifications of the products it makes while resellers and comparison services republish copies of them. A query over a product's specification is answered from the manufacturer alone. However, if any copies of, different, or wrong specifications exist in the federation, they are considered equally 'true'.

We will work towards a query execution model that resolves this observation while also allowing the delegation of queries *during* query execution. First we introduce the concept of a *composite source*, an aggregating view over sources in $\mathbb{D}$ which contains no data of its own.

Definition 7 (Composite source)**.** Fix a setting $W = \langle \mathbb{D}, \text{data}, \mathcal{D}_0, \text{disc} \rangle$. A source $c \in \mathbb{D}$ is *composite* when it holds no data of its own and is instead given by a set $\mathcal{D}_c \subseteq \mathbb{D}$ of sources it aggregates. It contains all of their data,

$$\text{data}(c) = \bigcup_{d \in \mathcal{D}_c} \text{data}(d),$$

and a request to c is evaluated over all of it. Sources that are not composite are *base* sources. We require the aggregation relation to be well-founded such that no source aggregates itself either directly or through a chain. Thus, when unfolding $\text{data}(c)$ it is expressed in base sources.

In the case $\mathcal{D}_c = \{d\}$ with d a base source, $\text{data}(c) = \text{data}(d)$. Then, c is equivalent to d.
{TODO: Add this to after claim is defined} and interchangeable when c claims (Def. 8) nothing.

Next we introduce *claims*, which declare a subset of data the composite source aggregates over that matches a certain criterion.

Definition 8 (Claim)**.** A composite source c may *declare* a claim. A claim is a *triple-wise* criterion $\sigma_c : \mathcal{T} \to \{0, 1\}$, an indicator function mapping triples to claim membership.

A claim is published with c and thus observed once c is retrieved. The core idea on which our query execution model builds is the *principle of exclusive claims*: queries are answered as if every claim were exclusive, that is, as if no source other than c held a triple claimed by c. Like the closed-world assumption, this does not state how the federation environment is; it fixes how we answer queries over it. The data need not satisfy it, and on the Web it often does not. Definition 10 makes the principle precise by fixing the answer on any data, and the engine enforces that answer, which we discuss in Section {TODO: ?}

Claims indicate that a composite source is the only source for answers for triples that matched the claim indicator function. Engines can then reason about this claim to determine for what data delegating BGPs to composite sources declaring a claim is correct. As an example, in the product catalogue federation, the manufacturer declares that statements about its products are to be taken from it alone, so that copies held by resellers are disregarded.

Nothing so far prevents a source from claiming data it should not be allowed to. On the Web, any source could claim authority over, for example, DBpedia without being affiliated with it, and thereby drop every DBpedia result held elsewhere. We therefore bound each claim by what its source is entitled to.

Definition 9 (Admissibility, effective claim)**.** An *admissibility function* assigns every composite c a criterion $\rho_c : \mathcal{T} \to \{0,1\}$, the triples c is entitled to claim. It is part of the setting, not of any declaration, and depends only on the triple and the identity of c. The *effective claim* of c is

$$\alpha_c(t) \;=\; \sigma_c(t) \cdot \rho_c(t),$$

with $\alpha_d \equiv 0$ for base sources and for composites that declare nothing. A source d *claims* t when $\alpha_d(t) = 1$, and the principle of exclusive claims is read with effective claims.

An illegitimate claim thus shrinks to its admissible part rather than being rejected whole: a composite claiming both DBpedia and its own namespace keeps the latter.

Definition 10 (Claim semantics)**.** A triple $t \in \mathrm{data}(d)$ is *admitted from* d if

$$\alpha_d(t) = 1 \quad \text{or} \quad \alpha_c(t) = 0 \text{ for all } c \in \mathcal{D} \setminus \{d\}, \tag{2}$$

and for $A \subseteq \mathcal{D}$ the *claim-respecting graph* is

$$G_A^{\alpha} \;=\; \bigcup_{d \in A} \{\, t \in \mathrm{data}(d) \mid t \text{ is admitted from } d \,\}.$$

The *claim semantics* of B is $[\![B]\!]_{G_{\mathcal{D}}^{\alpha}}$. It replaces the reference answer of Definition 2, and correctness of a delegation (Definition 4) is measured with G_A^{α} in place of G_A.

Thus, any claimed triple only occurs within the source that claims it, while unclaimed triples are left in the data as-is. If no source claims anything, $G_A^{\alpha} = G_A$ and the claim semantics is the reference answer of Definition 2.

{**TODO**: Possibly tie to literature here about subwebs and relational?}

We keep admissibility abstract until Section 3.6, where we instantiate it by namespaces; the results in between hold for any admissibility function.

Assumption 2 (Disjoint claims)**.** *No triple is claimed by two sources: for $c \neq c'$ and every $t \in \mathcal{T}$, $\alpha_c(t) \cdot \alpha_{c'}(t) = 0$.*

Section 3.6 gives a sufficient condition for namespace claims. We discuss relaxing the assumption in Section 3.8.1. {**TODO**: What happens if we relax this?}

3.4 Delegating Conjunctive Queries

Under this query execution model, we now show that correct delegation to composite sources can be established under *incomplete* discovery.

Lemma 2 (Claimed triples occur only in their claimant)**.** *Under Assumption 2, let $c \in \mathcal{D}$ claim τ. Then τ is admitted from no source $d \neq c$. Consequently, for every $A \subseteq \mathcal{D}$, $\tau \in G_A^{\alpha}$ implies $c \in A$ and $\tau \in \mathrm{data}(c)$.*

Proof. Suppose τ is admitted from some $d \neq c$. The second clause of (2) fails, as $c \in \mathcal{D} \setminus \{d\}$ claims τ. So the first holds and d claims τ, contradicting Assumption 2. □

A delegated composite answers over all of $\mathrm{data}(c)$, so its answer must be filtered to what the claim semantics admits. The engine can always check the claim of c, as α_c is known once c is retrieved. For a BGP B' we therefore keep the solutions whose matched triples c all claims,

$$\Phi_c(\Omega) \;=\; \big\{\, \mu \in \Omega \;\big|\; \alpha_c(\tau) = 1 \text{ for every } \tau \in \mu(B') \,\big\}.$$

Theorem 1 shows that the condition of Proposition 1 cannot be checked under incomplete discovery, as a source behind the frontier may hold triples that match B'. For triples that a composite c claims, this cannot happen. By Lemma 2, such a triple is admitted only from c, so no other source contributes it to $G = G_{\mathcal{D}}^{\alpha}$, whether it has been discovered or not. Filtering the answer of c with Φ_c thus yields exactly the solutions of B' that consist of triples c claims. We take the remaining solutions from the rest of the plan, which evaluates B' as usual and drops what c already supplies. With

$$\overline{\Phi}_c(\Omega) \;=\; \Omega \setminus \Phi_c(\Omega)$$

the solutions μ with at least one triple of $\mu(B')$ that c does not claim, let

$$R^* \;:=\; \Phi_c\big([\![B']\!]_{\mathrm{data}(c)}\big) \;\cup\; \overline{\Phi}_c\big([\![B']\!]_G\big).$$

Theorem 2 (Delegation to a claim)**.** *Under Assumption 2, let $c \in \mathcal{D}(T)$ be a composite and $B' \subseteq B$ non-empty. Then*

$$\Phi_c\big([\![B']\!]_{\mathrm{data}(c)}\big) \;=\; \Phi_c\big([\![B']\!]_G\big),$$

and this equality is determined at T. Consequently, $R^* = [\![B']\!]_G$ with disjoint terms, so delegating B' to c is correct for every B' and produces no solution twice.

Proof. Write $H_c := \{\, \tau \in \mathcal{T} \mid \alpha_c(\tau) = 1 \,\}$ for the triples c claims. For any graph G', a solution $\mu \in [\![B']\!]_{G'}$ is kept by Φ_c exactly when $\mu(B') \subseteq G' \cap H_c$, so by (1)

$$\Phi_c\big([\![B']\!]_{G'}\big) \;=\; [\![B']\!]_{G' \cap H_c}.$$

Applying this to both sides, it suffices to show $\mathrm{data}(c) \cap H_c = G \cap H_c$. ($\subseteq$) A triple of $\mathrm{data}(c)$ that c claims is admitted from c by the first clause of (2), and $c \in \mathcal{D}$, so it lies in G. ($\supseteq$) A triple of G that c claims lies in $\mathrm{data}(c)$ by Lemma 2.

The left-hand side depends only on $\mathrm{data}(c)$ and α_c, both observed once c is retrieved. As the argument holds in every state satisfying Assumption 2, the claimed fragment of the reference answer is known at T.

For the consequence, substituting the equality into R^* gives

$$R^* \;=\; \Phi_c\big([\![B']\!]_G\big) \cup \overline{\Phi}_c\big([\![B']\!]_G\big) \;=\; [\![B']\!]_G,$$

as Φ_c and $\overline{\Phi}_c$ split $[\![B']\!]_G$ into two disjoint parts, which are the two terms of R^*. A BGP evaluates to the join of its patterns, and $\bowtie$ is associative and commutative, so $B' \subseteq B$ gives

$$[\![B]\!]_G \;=\; [\![B']\!]_G \bowtie [\![B \setminus B']\!]_G \;=\; R^* \bowtie [\![B \setminus B']\!]_G.$$

This is the correctness condition of Definition 4, with R^* as the answer to B' and the claim semantics as reference (Definition 10). □

Theorem 2 recovers Proposition 1 under incomplete discovery. Restricted to the triples c claims, c is the only relevant source, by construction rather than by observation. The adversary of Theorem 1 can still place triples behind the frontier, but a triple that c claims is not admitted from any other source, so it cannot add solutions to the claimed fragment. These solutions also never need to be retracted. Every triple of $\mu(B')$ is held by its claimant, so it is covered (Definition 12), and the argument of Lemma 3 applies.

Correctness thus holds for every B'. What delegation saves, and how the base plan recognises results that c already supplies, depends on the admissibility function. Section 3.6.2 settles this for namespace claims.

3.5 Monotonicity

In classical discovery-based querying, a result is sound the moment it is produced: as $\mathcal{D}(T)$ grows, so does $G_{\mathcal{D}(T)}$, and BGP evaluation is monotone. Claims break this, as discovering a composite can disregard triples that were already used. We make this precise for the answer an engine can compute at time T, which can only take the claims of retrieved composites into account.

Definition 11 (Answer at time T)**.** For a discovery state $W(T)$, with claims under α, let

$$G^T = \bigcup_{d \in \mathcal{D}(T)} \{\, t \in \mathrm{data}(d) \mid d \text{ claims } t \text{ or no } c \in \mathcal{D}(T) \text{ claims } t \,\},$$

and $\Omega(T) = [\![B]\!]_{G^T}$. The answer is *monotone* if $\Omega(T) \subseteq \Omega(T')$ whenever $T \leq T'$.

Once discovery is complete, $\mathcal{D}(T) = \mathcal{D}$ by Lemma 1 and $G^T = G^\alpha_{\mathcal{D}}$, so $\Omega(T)$ is the answer under claims. Without claims, $G^T = G_{\mathcal{D}(T)}$ and Ω is monotone.

Proposition 2 (Claims break monotonicity)**.** *There is a setting with times $T \leq T'$ such that $\Omega(T) \not\subseteq \Omega(T')$.*

Proof. Take $B = \{(?s, p, ?o)\}$ and a composite c that claims $\tau = (s, p, o)$. Let a base source $d \notin \mathcal{D}_c$ hold τ while $\tau \notin \mathrm{data}(c)$, and let d be retrieved before c. If $\mathcal{D}(T) = \{d\}$, then $\mu = \{?s \mapsto s,\ ?o \mapsto o\} \in \Omega(T)$. Once $c \in \mathcal{D}(T')$, c claims τ, so τ is no longer admitted from d, nor from c as $\tau \notin \mathrm{data}(c)$. Hence $\mu \notin \Omega(T')$. □

The triple τ is claimed by c but held only elsewhere; under namespace claims it is a statement by d about a resource in the namespace of c. It is exactly the kind of triple the adversary of Theorem 1 places behind the frontier, and exactly what Theorem 2 needs disregarded. Non-monotonicity is thus the price of delegation under incomplete discovery rather than an artefact of our definitions. **{TODO:** Formalise as a necessity result?**}** It is, however, confined to such triples.

Definition 12 (Covered triple)**.** A triple t is *covered* if every composite $c \in \mathcal{D}$ that claims t also holds it, $t \in \mathrm{data}(c)$.

Lemma 3 (Covered solutions are stable)**.** *Under Assumption 2, let $\mu \in \Omega(T)$ be such that every triple of $\mu(B)$ is covered. Then $\mu \in \Omega(T')$ for every $T' \geq T$, and $\mu \in [\![B]\!]_{G^\alpha_{\mathcal{D}}}$.*

Proof. Let $\tau \in \mu(B)$. As $\mu(B) \subseteq G^T$ by (1), some $d \in \mathcal{D}(T)$ holds τ. By Assumption 2 at most one source claims τ. If no source of $\mathcal{D}(T')$ claims it, τ is admitted at T' from $d \in \mathcal{D}(T')$. Otherwise its claimant $c \in \mathcal{D}(T')$ holds τ, as τ is covered, and τ is admitted from c. Hence $\mu(B) \subseteq G^{T'}$, and $\mu \in \Omega(T')$ by (1). The same argument with $\mathcal{D}$ in place of $\mathcal{D}(T')$ gives $\mu \in [\![B]\!]_{G^\alpha_{\mathcal{D}}}$. □

Which triples are covered depends on admissibility. For namespace claims it follows from where a triple is stored, which Section 3.6 uses to resolve claim status in practice.

3.6 Subject-Namespace Claims

The results so far hold for any admissibility function satisfying Assumption 2. Two further questions need more structure: when a sub-query whose patterns have variable subjects can be delegated, and how an engine settles claim status in practice. Both rely on claims that are decided by the subject of a triple alone.

Definition 13 (Subject-determined claims)**.** Claims are *subject-determined* if for every composite c there is a set of terms S_c such that $\alpha_c(t) = 1$ iff $\mathrm{subj}(t) \in S_c$.

For this work we instantiate admissibility by namespaces, as discovery-based querying on the Web identifies sources by IRI. With $\mathrm{ns}(c)$ the namespace of the IRI of c,

$$\rho_c(t) = 1 \iff \mathrm{subj}(t) \in \mathrm{ns}(c),$$

and a composite that declares claims its whole namespace, $\sigma_c = \rho_c$. Claims are then subject-determined with $S_c = \mathrm{ns}(c)$. A composite with namespace http://example.org/ thus claims every triple whose subject starts with it. ρ_c depends only on the identity of c, so the engine knows it as soon as it discovers c, whatever lies behind the frontier. Admissibility bounds what c may *claim*, not what it may *aggregate*; for the latter, write $\mathrm{iri}(d)$ for the IRI of a source d.

Assumption 3 (Namespace aggregation)**.** *A composite aggregates every source in its namespace: if $\mathrm{iri}(d) \in \mathrm{ns}(c)$ then $d \in \mathcal{D}_c$.* **{TODO:** *align with subweb paper***}**

The picture is that of horizontal partitioning, with each composite the sole source of record for a region of the subject space. The difference is that the partitioning is declared by the participants rather than imposed by a coordinator, so no global catalogue of it exists and both the participants and the partitioning are discovered during execution. Assumption 2 holds when no composite's namespace extends another's. Disjoint aggregation alone does not suffice: a composite on http://example.org/ and one on http://example.org/a/ may aggregate disjoint documents and still both claim a triple about http://example.org/a/x.

3.6.1 Resolving claim status. Call $t \in \mathrm{data}(d)$ *local* if no composite namespace separates its subject from its source: for every composite c, $\mathrm{subj}(t) \in \mathrm{ns}(c)$ implies $\mathrm{iri}(d) \in \mathrm{ns}(c)$. Under Assumption 3 a local triple is covered, since a composite claiming it has $\mathrm{subj}(t) \in \mathrm{ns}(c)$, hence $d \in \mathcal{D}_c$ and $t \in \mathrm{data}(c)$. By Lemma 3, solutions built from local triples are sound as soon as they are produced; only solutions using a non-local triple, a statement about a resource in another namespace, can be invalidated by later discovery.

Remark 1 (Resolving claim status by request). In practice the engine need not wait for discovery to settle whether a non-local triple t is claimed. By Assumption 2 at most one composite can claim t, and under namespace admissibility its location is fixed by $\mathrm{subj}(t)$ alone: it is the composite, if any, whose namespace contains $\mathrm{subj}(t)$. The engine can therefore resolve the claim status of t with a request to this possible aggregator, for example by looking up an advertised composite along the path of $\mathrm{subj}(t)$ and, if one claims t, asking it whether it holds t. Including these lookups in disc makes the aggregator part of $\mathrm{reach}_B(W)$, so the outcome agrees with the answer under claims. An engine that resolves the status of every non-local triple before using it emits only solutions that remain valid, at the

cost of one lookup per foreign namespace and one membership check per non-local triple, rather than a wait for the frontier to empty. This mirrors authoritative reasoning, where authority is decided per term by dereferencing [7]. {**TODO**: Mechanism for advertising a composite, and for establishing that none exists: storage description, link header?}

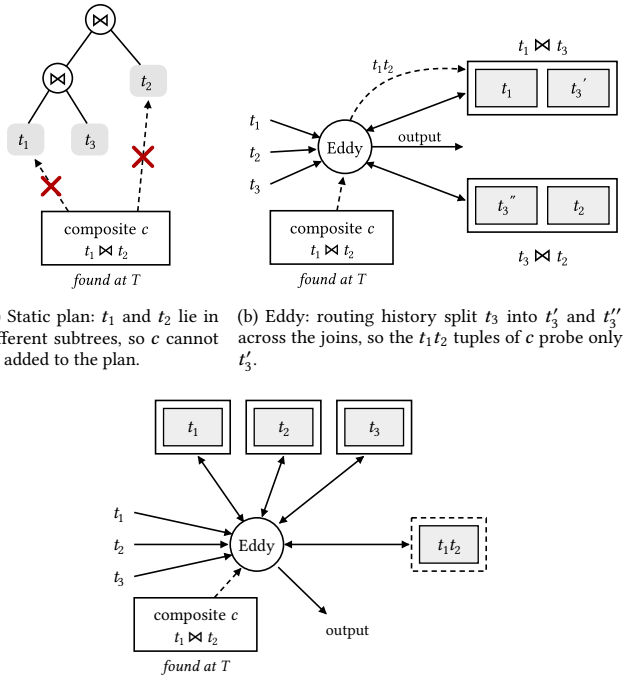
3.6.2 Star- and chain-shaped sub-queries. Theorem 2 makes delegation correct for every B'. What it saves depends on the base plan, the second term of R^*. Results that c already supplies are discarded by $\overline{\Phi}_c$, but only after the base plan has produced them. Delegation pays off when the base plan recognises such results early, or does not need the triples c claims at all. The latter holds for sub-queries without mixed solutions.

Definition 14 (Claim-uniform sub-query)**.** Let $G = G^{\alpha}_{\mathcal{D}}$. A BGP B' is *claim-uniform* for a composite c if every $\mu \in [\![B']\!]_G$ has c claim either every triple of $\mu(B')$ or none of them.

For a claim-uniform B', a solution with one triple that c does not claim uses no triple that c claims, so the base plan may disregard the claim of c for the patterns of B'.

Subject-determined claims make the claim status of a matched triple a function of the binding of its subject. For a star whose patterns share the subject variable $?s$, all triples matched by a solution μ have subject $\mu(?s)$, so either c claims all of them or none. The star is therefore claim-uniform for every composite, whatever the data, and Φ_c keeps exactly the solutions with $\mu(?s) \in S_c$. Patterns with distinct subject variables, as in chains, admit mixed solutions.

{**TODO**: Formalise when the base plan recognises results that c already supplies, under subject-determined claims. Considerations. (1) A solution μ is fully claimed by c iff every subject of $\mu(B')$ lies in S_c. Its claim status is thus decided by any sub-pattern $P \subseteq B'$ whose variables include all subject variables of B' (anchor variables). Constant subjects are known upfront. (2) Asymmetry. "Not fully claimed" is decided as soon as one bound subject lies outside S_c, so such partial results are kept without further checks. "Fully claimed" needs all subject variables bound. (3) Star. One subject variable, so decided at the first pattern. Claim-uniform, so the base plan never needs the claimed triples of B' and fetches nothing from the namespace of c for B'. This needs unclaimed triples held by c to be found through the sources it aggregates (formerly Assumption "reachable aggregation", $\mathcal{D}_c \subseteq \mathcal{D}$). (4) Chain of size 2. The first pattern binds both subject variables, so also decided at the first pattern. Not claim-uniform ($?s$ in S_c, $?x$ outside), so the claimed triples are still needed. Saves join work, not fetches. (5) Chain of size n. Decided only after the first $n-1$ patterns ($n-2$ joins along the chain), so little can be pruned. Stars and size-2 chains are therefore the attractive candidates (star-first heuristic, Section 3.7). (6) Where the base plan gets claimed triples from. They are admitted only from c (Lemma 2). Local triples can be read from documents in $\mathcal{D}_c$ (Assumption 3), non-local ones need the lookup of Remark 1. (7) Tie $\overline{\Phi}_c$ to routing, as the filter on the base copy that declines c (Section 3.7.1).}



(a) Static plan: t_1 and t_2 lie in different subtrees, so c cannot be added to the plan.

(b) Eddy: routing history split t_3 into t_3' and t_3'' across the joins, so the t_1t_2 tuples of c probe only t_3'.

(c) Eddy over SteMs: each pattern has a single SteM, so c becomes a new input whose t_1t_2 tuples get a SteM of their own.

Figure 1: Integrating a composite source c, discovered at time T and answering $t_1 \bowtie t_2$, into a running plan for $t_1 \bowtie t_2 \bowtie t_3$. Dashed elements appear at T.

3.7 Adaptive Query Planning with Composite Sources

Theorem 2 shows that a sub-query can be delegated correctly at any time. We now turn to *how* an engine uses such delegations. Composite sources are discovered while the query runs, so the engine must integrate a composite's answers into a plan that is already executing and already holds intermediate state.

A static plan fixes its join tree before execution. A composite answering $t_1 \bowtie t_2$ cannot be used when t_1 and t_2 lie in different subtrees, since no operator consumes their joined result (Figure 1a).

Eddies [1] replace the fixed join order by per-tuple routing, but each join operator keeps the tuples routed to it, so the state of a single pattern is split across operators by past routing decisions [5]. In Figure 1b, t_3 is split into t_3' and t_3'', and no operator holds all of t_3 for the t_1t_2 tuples of c to probe. Keeping every delegatable sub-query within one subtree avoids this, but commits the plan in advance to composites that may never be found, and overlapping candidates such as $t_1 \bowtie t_2$ and $t_2 \bowtie t_3$ cannot both be kept together. STAIRS [5] lifts the restriction by migrating state between operators, at the cost of a migration whenever a new composite is discovered.

Alternatively, an engine can run several plans in parallel [8, 15, 16]: a default plan beside plans that join delegatable sub-queries first. These plans are chosen before execution, whereas the sub-query best delegated to a composite depends on that composite

and is known only once it is found. As different composites may favour overlapping sub-queries, the required set of plans grows combinatorially and cannot be fixed upfront.

State Modules (SteMs) [11] remove the dependence on routing history. A SteM holds the tuples of a single input and supports *build* and *probe*; a join becomes two independent half-joins, and intermediate results are not stored. Since each input has exactly one SteM, its tuples are reachable however earlier tuples were routed. A composite discovered at runtime simply becomes a new input (Figure 1c): its t_1t_2 tuples are built into a new SteM and probe the complete SteM of t_3, and t_3 tuples arriving later probe the new SteM. No state is moved and no plan is rebuilt; the price is that intermediate results are recomputed rather than cached.

3.7.1 Routing. A composite c gives the patterns of its sub-query $B_c \subseteq B$ a second source: a partial result may take them from the SteM of c or from the base SteMs of B_c. The eddy explores both by routing a copy of the tuple along each, and the copies must stay apart, so that every pattern of a result is answered by exactly one operator. Each composite adds such a split, thus composites with overlapping sub-queries should stay separate, while disjoint composites may occur on the same routing path. Rather than materialising a plan per combination of composites, we keep a single eddy and let each tuple carry the constraints of the choices made for it.

Exclusion masks. Each operator o is labelled with the patterns it answers, $B_o = \{t\}$ for the SteM of t and $B_o = B_c$ for a composite, stored as a bitmask over B. Every tuple carries a mask M of the patterns it has answered and a mask M_c to indicate if the tuple has 'commited' to a composite during a split. This constrains the routing to all operators B_o that satisfy

$$(M \cup M_c) \cap B_o = \emptyset, \forall B_o \notin B_c$$

Copies share their route for as long as possible and split lazily: the copy bound for c as indicated by M_c may pass through SteMs outside B_c before reaching c, and at the split it is stamped $M \leftarrow M \cup B_c$, while the copy that keeps the base route gives up c.

This single test keeps the copy away from the base SteMs that c replaces and from every composite c' with $B_{c'} \cap B_c \neq \emptyset$, so overlapping composites are mutually exclusive. A composite discovered at runtime adds one operator and its mask; no existing tuple, SteM or route changes. The lazy branching allows the router to fully adaptively integrate the composite resource, without requiring the composite source to be joined exactly when the split happens in routing.

Canonical order. Copies may still reach the same composites, e.g. c_a then c_b and c_b then c_a. To prevent this, at a split the base copy declines every composite offered there, and the copy bound for c_j declines those with a lower discovery index. Two copies of a split thus differ on a composite only one of them joins, so each combination of disjoint composites is produced exactly once.
{**TODO**: Proposition: the eddy emits every solution of $[\![B]\!]_{G_{\mathcal{D}}^{\alpha}}$ exactly once.}

Practical Implementation. We follow the table-based routing implementation [4] where routing is not decided per tuple. Instead the tuple is looked up in a routing table indexed by its mask M, and

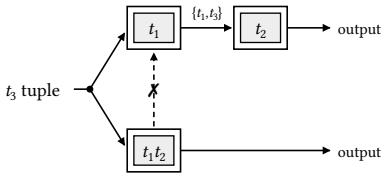


Figure 2: Parallel routing for $t_1 \bowtie t_2 \bowtie t_3$ with a composite c answering $t_1 \bowtie t_2$. A t_3 tuple splits into a copy that probes the base SteMs and a copy bound for c, stamped with B_c, which bars it from the SteMs that c replaces.

the routing policy reorders this table only every k tuples. Adding a composite c turns each entry with $M \cap B_c = \emptyset$ into a list of alternative routes, the new route holding c and the operators disjoint from both M and B_c. Whenever a table entry with two *distinct* next operator routes is encountered, the tuple is pushed to each operator, splitting the join plan. As masks range over the patterns of B rather than over operators, a composite adds routes but no states, and the table keeps at most $2^{|B|}$ entries.

3.8 Correctness

3.8.1 Overlapping Sources. Correctness when doing overlapping sources is difficult blabla, cases: Overlapping domain, overlapping triple patterns, both, neither. Given the time dimension, how do we decide what is possible

3.9 Discovery

{**TODO**: Do I even want discovery to be a topic as its specific to the environment}

4 System Application

To evaluate the performance impact of delegating sub-queries we implement hybrid federated link traversal-based query processing. Specifically, within solid blabla

Note this is a concrete application of the stuff introduced here, which is not limited to Solid and link traversal.
{**TODO**: Mention this paper as someone who already uses stems: https://arxiv.org/pdf/1607.01046}

5 Evaluation

TODO: Ensure correctness of multiple composite resources!! Experiments: - Full benchmark and abalations - Client-scaling test (find server-side usage and client-side scaling behavior) - Test query with increasing pod depth to find answer - Query where linear is fastest to delegate with single pod (show star-only fails and cost-based fails) - Query where linear is fastest to delegate with multiple pods (show if cost-based doesn't fail) - Query where star is fastest multiple pods (show if cost-based doesn't fail) - Multiple vs Single pod query

6 Conclusion

Summarize results and point to future work.

References

[1] Ron Avnur and Joseph M Hellerstein. 2000. Eddies: Continuously adaptive query processing. In *Proceedings of the 2000 ACM SIGMOD international conference on Management of data*. 261–272.

[2] Sijin Cheng and Olaf Hartig. 2020. FedQPL: a language for logical query plans over heterogeneous federations of RDF data sources. In *Proceedings of the 22nd International Conference on Information Integration and Web-based Applications & Services*. 436–445.

[3] Richard Cyganiak, David Wood, and Markus Lanthaler. 2014. *RDF 1.1 Concepts and Abstract Syntax*. W3C Recommendation. W3C. https://www.w3.org/TR/rdf11-concepts/

[4] Amol Deshpande. 2004. An initial study of overheads of eddies. *ACM SIGMOD Record* 33, 1 (2004), 44–49.

[5] Amol Deshpande and Joseph M Hellerstein. 2004. Lifting the burden of history from adaptive query processing. In *Proceedings of the Thirtieth international conference on Very large data bases-Volume 30*. 948–959.

[6] Steve Harris and Andy Seaborne. 2013. *SPARQL 1.1 Query Language* (1.1 ed.). W3C Recommendation. W3C. https://www.w3.org/TR/sparql11-query/ W3C Recommendation 21 March 2013.

[7] Aidan Hogan, Andreas Harth, and Axel Polleres. 2009. Scalable Authoritative OWL Reasoning for the Web. *International Journal on Semantic Web and Information Systems* 5, 2 (2009), 49–90. doi:10.4018/jswis.2009040103

[8] David Justen, Daniel Ritter, Campbell Fraser, Andrew Lamb, Allison Lee, Thomas Bodner, Mhd Yamen Haddad, Steffen Zeuch, Volker Markl, and Matthias Boehm. 2024. POLAR: adaptive and non-invasive join order selection via plans of least resistance. *Proceedings of the VLDB Endowment* 17, 6 (2024), 1350–1363.

[9] Jorge Pérez, Marcelo Arenas, and Claudio Gutierrez. 2009. Semantics and complexity of SPARQL. *ACM Transactions on Database Systems (TODS)* 34, 3 (2009), 1–45.

[10] Eric Prud'hommeaux and Carlos Buil-Aranda. 2013. *SPARQL 1.1 Federated Query*. W3C Recommendation. W3C. https://www.w3.org/TR/sparql11-federated-query/

[11] Vijayshankar Raman, Amol Deshpande, and Joseph M Hellerstein. 2003. Using state modules for adaptive query processing. In *Proceedings 19th International Conference on Data Engineering*. IEEE, 353–364.

[12] Andreas Schwarte, Peter Haase, Katja Hose, Ralf Schenkel, and Michael Schmidt. 2011. FedX: Optimization Techniques for Federated Query Processing on Linked Data. In *The Semantic Web – ISWC 2011 (LNCS, Vol. 7031)*. Springer, 601–616. doi:10.1007/978-3-642-25073-6_38

[13] Audun Stolpe. 2015. A logical characterisation of SPARQL federation. *Semantic Web* 6, 6 (2015), 565–584. doi:10.3233/SW-140160

[14] Audun Stolpe and Jonas Halvorsen. 2017. Distributed query processing in the presence of blank nodes. *Semantic Web* 8, 6 (2017), 1001–1021. doi:10.3233/SW-160250

[15] Kostas Tzoumas, Amol Deshpande, and Christian S Jensen. 2010. Sharing-aware horizontal partitioning for exploiting correlations during query processing. *Proceedings of the VLDB Endowment* 3, 1-2 (2010), 542–553.

[16] Ziyun Wei and Immanuel Trummer. 2024. ROME: Robust query optimization via parallel multi-plan execution. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–25.

A Proof of Proposition 1

Proof of Proposition 1. We prove the special case of Definition 4 with $S = \{d\}$, so that $\mathcal{D} \setminus S = \mathcal{D}'$, and we assume $B' \neq \emptyset$.

The argument rests on the fact that a *single* triple pattern distributes over a union of sources: for every t and every $A \subseteq \mathcal{D}$,

$$[\![t]\!]_{G_A} \;=\; \bigcup_{e \in A} [\![t]\!]_{\mathrm{data}(e)}, \tag{3}$$

since matching one pattern involves no join. Note that (3) fails for BGPs, which is exactly what makes delegation non-trivial.

By hypothesis no source other than d is relevant to any pattern of B', so Definition 3 gives

$$[\![t']\!]_{\mathrm{data}(d')} = \emptyset \quad \text{for all } t' \in B',\ d' \in \mathcal{D}'. \tag{4}$$

Step 1. A BGP is the join of its patterns, and each pattern distributes over the sources by (3):

$$[\![B]\!]_{G_{\mathcal{D}}} \;=\; \bowtie_{t \in B} [\![t]\!]_{G_{\mathcal{D}}} \;=\; \bowtie_{t \in B} \bigcup_{e \in \mathcal{D}} [\![t]\!]_{\mathrm{data}(e)}.$$

Step 2. Since $\bowtie$ is associative and commutative, the delegated fragment splits off:

$$[\![B]\!]_{G_{\mathcal{D}}} \;=\; \Big(\bowtie_{t' \in B'} \bigcup_{e \in \mathcal{D}} [\![t']\!]_{\mathrm{data}(e)} \Big) \bowtie [\![B \setminus B']\!]_{G_{\mathcal{D}}}.$$

Step 3. As $\mathcal{D} = \mathcal{D}' \uplus \{d\}$, the inner union splits, and by (4) its $\mathcal{D}'$ part is empty:

$$\bigcup_{e \in \mathcal{D}} [\![t']\!]_{\mathrm{data}(e)} \;=\; \underbrace{\bigcup_{d' \in \mathcal{D}'} [\![t']\!]_{\mathrm{data}(d')}}_{=\,\emptyset} \cup\, [\![t']\!]_{\mathrm{data}(d)} \;=\; [\![t']\!]_{\mathrm{data}(d)}.$$

Substituting into Step 2 and reassembling the join,

$$[\![B]\!]_{G_{\mathcal{D}}} \;=\; [\![B']\!]_{\mathrm{data}(d)} \bowtie [\![B \setminus B']\!]_{G_{\mathcal{D}}}. \tag{5}$$

Step 4. It remains to identify R from Definition 4. Its second term vanishes, again by (3) and (4) and using $B' \neq \emptyset$:

$$[\![B']\!]_{G_{\mathcal{D}\setminus S}} \;=\; \bowtie_{t' \in B'} \bigcup_{d' \in \mathcal{D}'} [\![t']\!]_{\mathrm{data}(d')} \;=\; \bowtie_{t' \in B'} \emptyset \;=\; \emptyset.$$

With $S = \{d\}$ the first term of R is the single request to d, so $R = [\![B']\!]_{\mathrm{data}(d)} \cup \emptyset = [\![B']\!]_{\mathrm{data}(d)}$.

Conclusion. Substituting R into (5),

$$R \bowtie [\![B \setminus B']\!]_{G_{\mathcal{D}}} \;=\; [\![B]\!]_{G_{\mathcal{D}}},$$

which is the correctness condition of Definition 4. □